Boolean Parenthesization DP

This is actually the **most important part** of the Boolean Parenthesization DP. The formulas are not something to memorize—they come directly from the **truth tables** of the operators.

The intuition is:

> We are standing at an operator (`&`, `|`, `^`).
>
> We already know **how many ways** the left side can become **True** and **False**.
>
> We also know **how many ways** the right side can become **True** and **False**.
>
> Now we simply ask:
>
> **"Which combinations produce my desired answer (True or False)?"**

Let's derive every formula from scratch.

Step 1: Meaning of lT, lF, rT, rF

Suppose the expression is

```
Left  op  Right
```

After recursion we know

```
lT = ways Left becomes True
lF = ways Left becomes False

rT = ways Right becomes True
rF = ways Right becomes False
```

Imagine

```
lT = 2
lF = 3

rT = 4
rF = 5
```

This means

```
Left True   -> 2 ways
Left False  -> 3 ways

Right True  -> 4 ways
Right False -> 5 ways
```

Every left possibility can combine with every right possibility.

For example

```
Left True (2 ways)
Right False (5 ways)

Total combinations

= 2 × 5 = 10
```

That's why we multiply.

Case 1 : AND (&)

Truth table

Left	Right	Result
T	T	T
T	F	F
F	T	F
F	F	F

Look carefully.

There is **only ONE way** to get True.

```
T & T
```

Therefore

```
ways(True)

= lT * rT
```

That's exactly the code.

```
C++
if(isTrue)
    ways += lT*rT;
```

Why do we need False?

Now suppose we want

```
Left & Right = False
```

Look at the truth table.

False happens in

```
T F
F T
F F
```

These are **three different situations**.

Situation 1

```
T & F
```

Ways

```
lT * rF
```

Situation 2

```
F & T
```

Ways

```
lF * rT
```

Situation 3

```
F & F
```

Ways

```
lF * rF
```

Since **any one** of these gives False,

we add them.

```
False

=
lT*rF
+
lF*rT
+
lF*rF
```

Exactly the code

```
C++
ways += lF*rT
      + lT*rF
      + lF*rF;
```

Intuition

Think of AND as

> **"Everybody must agree."**

Even **one False** ruins everything.

```
T & F -> False

F & T -> False

F & F -> False
```

Only

```
T & T
```

survives.

Case 2 : OR (|)

Truth table

Left	Right	Result
T	T	T
T	F	T
F	T	T
F	F	F

Now ask

Which combinations give True?

Answer

```
T T

T F

F T
```

Therefore

```
True

=
lT*rT
+
lT*rF
+
lF*rT
```

Exactly the code.

Why only one False case?

Look again.

Only

```
F F
```

gives False.

So

```
False

=
lF*rF
```

That's why

```
C++
else
    ways += lF*rF;
```

Intuition

Think of OR as

> **"At least one person should say yes."**

If even one side is True

```
T F
F T
T T
```

Result becomes True.

Only

```
F F
```

fails.

Case 3 : XOR (^)

Truth table

Left	Right	Result
T	T	F
T	F	T
F	T	T
F	F	F

XOR means

> **Exactly one side should be True.**

Which combinations give True?

```
T F
F T
```

Therefore

```
True
=
lT*rF
+
lF*rT
```

Exactly the code.

Why False?

False happens when

both are equal.

```
T T
F F
```

Therefore

```
False
=
lT*rT
+
lF*rF
```

Exactly

```
C++
ways += lT*rT
      + lF*rF;
```

Intuition

Think of XOR as

> **"Different is True, Same is False."**

```
T T -> same -> False
F F -> same -> False
T F -> different -> True
F T -> different -> True
```

The easiest way to derive these formulas in an interview

Forget the formulas.

Just write the truth table.

For example, for `&`

```
T T -> T
T F -> F
F T -> F
F F -> F
```

If you want **True**

collect all rows giving True.

```
T T
Answer
lT*rT
```

If you want **False**

collect all False rows.

```
T F
F T
F F
Answer
lT*rF
+
lF*rT
+
lF*rF
```

Do exactly the same for every operator.

A simple memory trick

```
AND (&)
True  -> both True
False -> anything else
```

```
OR (|)
False -> both False
True -> anything else
```

```
XOR (^)
True -> different
False -> same
```

This single mnemonic is enough to reconstruct all six formulas during an interview, so you never need to memorize them.

Excellent question. This is actually the point where most people get confused the first time they learn this DP.

The confusion is:

> **"If my final answer is asking for True (`isTrue = 1`), why am I even computing `lF` and `rF`? Why don't I only compute True?"**

The answer is:

> **Because the operator may need one side to be False in order to produce True.**

Let's build the intuition.

Imagine this expression

```
F ^ T
```

Suppose we're evaluating

```
Left ^ Right
```

and we want the result to be **True**.

Now ask yourself:

Can XOR become True if both sides are True?

```
T ^ T = False
```

No.

Can it become True if both are False?

```
F ^ F = False
```

No.

The only possibilities are

```
T ^ F
F ^ T
```

See what happened?

To make the **whole expression True**, **one side actually has to be False**.

So even though the final answer is True, we still need

```
Left False
Right False
```

counts.

Otherwise, we'd miss valid parenthesizations.

Another example

Suppose

```
(T|F) ^ (F&T)
```

Imagine recursion tells us

```
Left can become True  = 5 ways
Left can become False = 3 ways

Right can become True = 2 ways
Right can become False = 7 ways
```

Now we are at

```
Left ^ Right
```

and we want True.

True comes from

```
T ^ F
```

Number of ways

```
5 × 7 = 35
```

and

```
F ^ T
```

Number of ways

```
3 × 2 = 6
```

Total

```
35 + 6 = 41
```

If you never computed

```
Left False
Right False
```

then where would the **6** come from?

You would lose those valid ways.

Now let's see OR

Truth table

```
T | T = T
T | F = T
F | T = T
F | F = F
```

Suppose we want True.

Notice something.

One valid case is

```
T | F
```

Another is

```
F | T
```

Again,

to produce **True**, OR sometimes **needs a False side**.

Therefore we require

```
lT
lF
rT
rF
```

because

```
T | F
```

uses

```
lT × rF
```

and

```
F | T
```

uses

```
lF × rT
```

Then why compute `lF * rF`?

This is the exact question you asked.

Suppose

```C++
f(i,j,0)
```

is called.

Notice this carefully.

The recursion is **not only solving**

```
How many ways to make this expression True?
```

It is also solving

```
How many ways to make this expression False?
```

The function is

```
C++
f(i,j,isTrue)
```

That means there are **two different problems**.

```
f(i,j,1)
```

↓

How many ways to become True?

and

```
f(i,j,0)
```

↓

How many ways to become False?

So eventually recursion **will** call

```
C++
f(i,j,0)
```

many times.

When that happens,

for OR,

False happens only when

```
F | F
```

Therefore

```
lF × rF
```

is required.

Example of recursion

Consider

```
T | F & T
```

Suppose you're evaluating

```
T | (F&T)
```

To know whether the whole expression becomes True,

you need to know

```
How many ways can (F&T) become True?
```

and

```
How many ways can (F&T) become False?
```

Because

```
T | True = True
T | False = True
```

So recursion calls

```
f(F&T,1)
and
f(F&T,0)
```

See?

Even while solving the **overall True answer**, we must know **both outcomes of every subexpression**.

Think of every subexpression as a machine

Instead of thinking

```
This expression becomes True.
```

think

```
This expression has
8 ways to become True
5 ways to become False
```

Every recursive call returns **one of these two numbers** depending on `isTrue`.

When combining two machines, the operator chooses which combinations it needs.

For example, for OR

```
Left machine
True : 8
False : 5
```

```
Right machine
True : 3
False : 4
```

Operator OR says

> "To make me True, I'll use"

```
8×3
8×4
5×3
```

Operator OR also says

> "To make me False, I'll use"

```
5×4
```

So **every operator needs to know both outputs (True and False) of its left and right children**, because different operators use different combinations to achieve the desired result.

The key intuition is:

The recursion is **not storing whether an expression is True or False**. It is storing:

- **How many ways** this subexpression can evaluate to **True**.
- **How many ways** this subexpression can evaluate to **False**.

When a parent operator combines two subexpressions, it may need either of those counts depending on the operator's truth table. That's why `lF` and `rF` are indispensable—even when the final goal is to count ways to evaluate the whole expression to **True**.

Yes, you're very close. There's just one important correction.

The rule is **not**:

> "For AND, we only need left True and right True."

Instead, the rule is:

> **For every operator (`&`, `|`, `^`), we always compute both True and False counts for the left and right subexpressions.**

The difference is **which of those counts are used** depending on whether we're trying to make the current expression True or False.

Let's see it clearly.

Suppose we're at

```
Left  op  Right
```

and recursion has already given us

```
lT = ways Left becomes True
lF = ways Left becomes False

rT = ways Right becomes True
rF = ways Right becomes False
```

Now the operator decides what it needs.

AND (&)

If we want **True**:

```
T & T = T
```

So we use only

```
lT × rT
```

But if we want **False**:

```
T & F = F
F & T = F
F & F = F
```

So we need

```
lT × rF
+
lF × rT
+
lF × rF
```

Notice that **AND also needs False counts**, just not when we're computing the **True** case.

OR (|)

If we want **True**:

```
T | T = T
T | F = T
F | T = T
```

So we need

```
lT × rT
+
lT × rF
+
lF × rT
```

If we want **False**:

```
F | F = F
```

So we need

```
lF × rF
```

XOR (^)

If we want **True**:

```
T ^ F = T
F ^ T = T
```

So we need

```
lT × rF
+
lF × rT
```

If we want **False**:

```
T ^ T = F
F ^ F = F
```

So we need

```
lT × rT
+
lF × rF
```

The most important intuition

Think of every subexpression as returning **two numbers**:

```
Expression

can become True  -> x ways

can become False -> y ways
```

The parent operator (`&`, `|`, or `^`) looks at its truth table and decides **which combinations of these two numbers it needs**.

So the recursion always computes **both** because a parent may ask for either one.

A good mental model

Imagine each recursive call returns a box:

```
        Expression

       ┌──────────┐
       │True  = 5 │
       │False = 3 │
       └──────────┘
```

The parent operator opens the left box and the right box and picks the combinations it needs according to its truth table.

This is why the recursive function is defined as:

```
C++
f(i, j, isTrue)
```

instead of only

```
C++
f(i, j)
```

because every subexpression must be able to answer **both** questions:

- "How many ways can you become True?"
- "How many ways can you become False?"

That's the core idea behind the Boolean Parenthesization DP.